Materi Perkuliahan: Fungsi Agregat (Aggregate Functions)

1. Apa itu Fungsi Agregat?

Fungsi agregat adalah fungsi yang melakukan penghitungan pada sekumpulan nilai dan mengembalikan **satu nilai tunggal**. Biasanya fungsi ini digunakan bersamaan dengan klausa GROUP BY, namun untuk tahap awal kita pelajari penggunaannya secara global.

2. Jenis-Jenis Fungsi Agregat Utama

a. COUNT

Digunakan untuk menghitung jumlah baris atau data dalam sebuah kolom.

- COUNT (*): Menghitung total semua baris termasuk yang bernilai NULL.
- COUNT (nama_kolom): Menghitung jumlah baris yang memiliki nilai (mengabaikan NULL).

b. SUM

Digunakan untuk menjumlahkan total nilai numerik pada sebuah kolom.

Penting: Hanya bisa digunakan pada tipe data angka (Integer, Decimal, Float).

c. AVG (Average)

Digunakan untuk menghitung nilai rata-rata dari sebuah kolom numerik.

d. MIN dan MAX

- MIN: Mencari nilai terkecil.
- MAX: Mencari nilai terbesar.

Bisa digunakan untuk data angka, tanggal (Date), maupun teks (String).

3. Implementasi Langsung

Kita akan menggunakan database KampusDB yang telah dibuat sebelumnya. Mari kita tambahkan kolom biaya_semester untuk praktek SUM dan AVG.

SQL:

```
ALTER TABLE mahasiswa ADD biaya_semester DECIMAL(10,2);

UPDATE mahasiswa SET biaya_semester = 5000000 WHERE nim IN ('220101',
'220103', '220105');
UPDATE mahasiswa SET biaya_semester = 4500000 WHERE nim IN ('220102',
'220104');
```

Contoh Query Agregat:

1. **Menghitung Total Mahasiswa:**

```
SELECT COUNT(*) AS total_mahasiswa FROM mahasiswa;
```

2. **Mencari IPK Tertinggi dan Terendah:**

```
SELECT MAX(ipk) AS ipk_tertinggi, MIN(ipk) AS ipk_terendah FROM
mahasiswa;
```

3. **Menghitung Rata-rata IPK Mahasiswa Informatika:**

```
SELECT AVG(ipk) AS rata_rata_ipk_it
FROM mahasiswa
WHERE jurusan = 'Informatika';
```

4. **Menghitung Total Pemasukan Biaya Semester:**

```
SELECT SUM(biaya_semester) AS total_pendapatan FROM mahasiswa;
```

Materi Perkuliahan: Pengelompokan Data (Group By & Having)

1. Konsep GROUP BY

Jika fungsi agregat (SUM, AVG, COUNT, dll) digunakan sendirian, mereka meringkas **seluruh tabel**. Namun, jika kita ingin melihat ringkasan **per kategori** (misal: rata-rata IPK *per jurusan*), kita wajib menggunakan GROUP BY.

Aturan: Setiap kolom yang muncul di SELECT tetapi tidak dikenakan fungsi agregat, **wajib** dicantumkan di dalam klausa GROUP BY.

```
Sintak:

SELECT kolom_kategori, FUNGSI_AGREGAT(kolom_nilai)
FROM nama_tabel
WHERE kondisi_baris -- (Opsional) Filter sebelum dikelompokkan
GROUP BY kolom_kategori
ORDER BY kolom_kategori;
```

2. Filter Kelompok dengan HAVING

Banyak mahasiswa keliru menggunakan WHERE untuk memfilter hasil agregat.

- **WHERE**: Memfilter baris data **sebelum** dikelompokkan.
- **HAVING**: Memfilter hasil kelompok **sesudah** agregasi dilakukan.

Analogi: > * "Tampilkan jurusan yang mahasiswanya berasal dari Jakarta" Gunakan **WHERE**.

"Tampilkan jurusan yang rata-rata IPK-nya di atas 3.0" Gunakan **HAVING**.

3. Implementasi Langsung (KampusDB)

a. Menghitung Jumlah Mahasiswa perJurusan

```
SELECT jurusan, COUNT(*) AS jumlah_mahasiswa
FROM mahasiswa
GROUP BY jurusan;
```

b. Mencari Rata-rata IPK per Semester yang Lebih dari 3.0

```
SELECT semester, AVG(ipk) AS rata_rata_ipk
FROM mahasiswa
GROUP BY semester
HAVING AVG(ipk) > 3.0;
```

Contoh:

1. Tampilkan total biaya semester yang didapat dari masing-masing jurusan.

```
SELECT jurusan, SUM(biaya_semester) AS total_pendapatan
FROM mahasiswa
GROUP BY jurusan;
```

2. Tampilkan jurusan yang memiliki lebih dari 2 mahasiswa.

```
SELECT jurusan, COUNT(*) AS total
FROM mahasiswa
GROUP BY jurusan
HAVING COUNT(*) > 2;
```

3. Tampilkan rata-rata IPK mahasiswa semester 4 per jurusan, tapi hanya untuk jurusan yang rata-ratanya di atas 3.2.

```
SELECT jurusan, AVG(ipk) AS rerata
FROM mahasiswa
WHERE semester = 4
GROUP BY jurusan
HAVING AVG(ipk) > 3.2;
```

Materi Perkuliahan: Subquery dan Nested Query

1. Subquery (disebut juga *Inner Query* atau *Nested Query*) adalah query SELECT yang berada di dalam perintah SQL lain (seperti SELECT, INSERT, UPDATE, atau DELETE).

- **Inner Query:** Query yang berada di dalam kurung, yang dieksekusi lebih dulu.
- **Outer Query:** Query utama yang menggunakan hasil dari Inner Query.

2. Jenis-Jenis Subquery Berdasarkan Hasilnya

a. Scalar Subquery: Subquery yang hanya menghasilkan **satu nilai tunggal** (satu baris dan satu kolom). Biasanya digunakan dengan operator perbandingan seperti =, <, >, atau !=.

Contoh: Tampilkan mahasiswa yang IPK-nya lebih tinggi dari rata-rata IPK seluruh mahasiswa.

```
SELECT nama, ipk
FROM mahasiswa
WHERE ipk > (SELECT AVG(ipk) FROM mahasiswa);
```

b. Multiple-Row Subquery: Subquery yang menghasilkan **satu kolom tetapi banyak baris**. Kita harus menggunakan operator khusus seperti IN, ANY, atau ALL.

Contoh: Tampilkan mahasiswa yang berada di jurusan yang sama dengan 'Andi'.

```
SELECT nama
FROM mahasiswa
WHERE id_jurusan IN (SELECT id_jurusan FROM mahasiswa WHERE nama =
'Andi');
```

c. Correlated Subquery: Ini adalah jenis yang lebih advance di mana *Inner Query* bergantung pada nilai dari *Outer Query*. Query ini dieksekusi berulang kali, sekali untuk setiap baris yang diproses oleh query utama.

Contoh: Tampilkan mahasiswa yang IPK-nya di atas rata-rata **jurusannya masing-masing**.

```
SELECT nama, id_jurusan, ipk
FROM mahasiswa m1
WHERE ipk > (SELECT AVG(ipk)
             FROM mahasiswa m2
             WHERE m2.id_jurusan = m1.id_jurusan);
```

3. Penempatan Subquery

Subquery tidak hanya bisa diletakkan di WHERE. Berikut adalah tempat lainnya:

- **Subquery di dalam FROM (Derived Table):** Menganggap hasil query sebagai tabel sementara.

```
SELECT MAX(rerata_ipk)
FROM (SELECT id_jurusan, AVG(ipk) AS rerata_ipk FROM mahasiswa GROUP BY
id_jurusan) AS tabel_rerata;
```

- **Subquery di dalam SELECT:** Menampilkan nilai tambahan di setiap baris.

```
SELECT nama, (SELECT nama_jurusan FROM jurusan WHERE id_jurusan =
m.id_jurusan) as jurusan
FROM mahasiswa m;
```

Contoh Kasus:

1. Cari nama pegawai yang memiliki gaji lebih besar dari gaji rata-rata departemen 'IT'.

```
SELECT nama FROM pegawai
WHERE gaji > (SELECT AVG(gaji) FROM pegawai WHERE departemen = 'IT');
```

2. Tampilkan data produk yang tidak pernah dipesan sama sekali (Gunakan NOT IN atau NOT EXISTS).

```
SELECT nama_produk FROM produk
WHERE id_produk NOT IN (SELECT DISTINCT id_produk FROM detail_pesanan);
```

3. Cari siapa mahasiswa yang memiliki IPK paling tinggi (Tanpa menggunakan LIMIT atau TOP, gunakan Subquery).

```
SELECT nama FROM mahasiswa
WHERE ipk = (SELECT MAX(ipk) FROM mahasiswa);
```

1. Operator EXISTS dan NOT EXISTS

Berbeda dengan IN yang membandingkan nilai, EXISTS hanya memeriksa apakah Subquery **menghasilkan baris atau tidak**. Jika ada setidaknya satu baris yang cocok, maka hasilnya TRUE.

- **Kapan digunakan?** Sangat efektif untuk pengecekan relasi antar tabel (seperti mencari data yang ada atau tidak ada pasangannya di tabel lain).
- **Performa:** Biasanya lebih cepat daripada IN untuk dataset besar karena database berhenti mencari begitu menemukan satu kecocokan.

Contoh Kasus: Tampilkan daftar jurusan yang sudah memiliki mahasiswa.

```
SELECT nama_jurusan
FROM jurusan j
WHERE EXISTS (
    SELECT 1 FROM mahasiswa m
    WHERE m.id_jurusan = j.id_jurusan
);
```

2. Operator ANY (atau SOME)

Operator ANY akan bernilai TRUE jika kondisi perbandingan terpenuhi oleh **salah satu saja** nilai yang dihasilkan oleh Subquery.

- = ANY sama fungsinya dengan IN.
- > ANY berarti "lebih besar dari nilai terkecil di dalam daftar".

Contoh Kasus: Cari pegawai yang gajinya lebih tinggi dari **salah satu** (siapa pun) pegawai di departemen 'Marketing'.

```
SELECT nama, gaji
FROM pegawai
WHERE gaji > ANY (SELECT gaji FROM pegawai WHERE departemen =
'Marketing');
```

3. Operator `ALL`

Operator `ALL` akan bernilai `TRUE` jika kondisi perbandingan terpenuhi oleh **seluruh** nilai yang dihasilkan oleh Subquery.

- `> ALL` berarti "lebih besar dari nilai tertinggi di dalam daftar".
- `!= ALL` sama fungsinya dengan `NOT IN`.

Contoh Kasus: Cari mahasiswa yang IPK-nya lebih tinggi dari **semua** mahasiswa di jurusan 'Teknik Elektro'.

```
SELECT nama, ipk
FROM mahasiswa
WHERE ipk > ALL (SELECT ipk FROM mahasiswa WHERE id_jurusan = 103);
```

Materi Perkuliahan: Penerapan SQL Join untuk Relasi Tabel

1. Konsep Dasar Relasi

Dalam database relasional, data dibagi ke dalam beberapa tabel untuk menghindari redundansi (Normalisasi). Tabel-tabel ini dihubungkan melalui:

- **Primary Key (PK):** Identitas unik dalam sebuah tabel.
- **Foreign Key (FK):** Kolom dalam suatu tabel yang merujuk pada Primary Key di tabel lain.

SQL Join adalah mekanisme untuk menggabungkan baris dari dua atau lebih tabel berdasarkan kolom yang terkait di antara mereka.

2. Jenis-Jenis SQL Join

a. INNER JOIN

Mengambil baris ketika ada kecocokan di **kedua** tabel. Jika data di satu tabel tidak memiliki pasangan di tabel lain, maka data tersebut tidak akan muncul.

Sintaks:

```
SELECT kolom
FROM Tabel_A
INNER JOIN Tabel_B ON Tabel_A.PK = Tabel_B.FK;
```

b. LEFT (OUTER) JOIN

Mengambil **semua** baris dari tabel kiri (Tabel A), dan baris yang cocok dari tabel kanan (Tabel B). Jika tidak ada kecocokan, hasil dari sisi kanan adalah NULL.

c. RIGHT (OUTER) JOIN

Kebalikan dari Left Join. Mengambil semua baris dari tabel kanan, dan baris yang cocok dari tabel kiri.

d. FULL OUTER JOIN

Mengambil semua baris ketika ada kecocokan di salah satu tabel (Kiri atau Kanan). *Catatan: Tidak semua DBMS (seperti MySQL) mendukung ini secara langsung, biasanya menggunakan UNION.*

3. Implementasi Studi Kasus: Sistem Perpustakaan

Mari kita terapkan pada dua tabel berikut:

Tabel buku

id_buku (PK)	judul	id_penerbit (FK)
1	Laskar Pelangi	10
2	Bumi Manusia	11
3	SQL Dasar	NULL

Tabel penerbit

id_penerbit (PK)	nama_penerbit
10	Bentang Pustaka
11	Lentera Dipantara
12	Gramedia

Contoh 1: Menampilkan Buku beserta Penerbitnya (Inner Join)

```
SELECT buku.judul, penerbit.nama_penerbit
FROM buku
INNER JOIN penerbit ON buku.id_penerbit = penerbit.id_penerbit;
```

Hasil: Buku 'SQL Dasar' tidak akan muncul karena id_penerbit-nya NULL.

Contoh 2: Menampilkan Semua Buku (Left Join)

```
SELECT buku.judul, penerbit.nama_penerbit
FROM buku
LEFT JOIN penerbit ON buku.id_penerbit = penerbit.id_penerbit;
```

Hasil: Semua buku muncul. 'SQL Dasar' akan memiliki nama_penerbit bernilai NULL.

4. Join Lebih dari Dua Tabel

Dalam aplikasi nyata, kita sering menggabungkan 3 tabel atau lebih.

Sintaks:

```
SELECT m.nama, j.nama_jurusan, f.nama_fakultas
FROM mahasiswa m
JOIN jurusan j ON m.id_jurusan = j.id_jurusan
JOIN fakultas f ON j.id_fakultas = f.id_fakultas;
```

Materi Perkuliahan: Implementasi Fungsi SQL dalam Operasi CRUD

1. Pendahuluan: Fungsi vs CRUD

Dalam siklus CRUD, fungsi SQL berperan sebagai "pengolah" data agar data yang disimpan atau ditampilkan sesuai dengan kebutuhan bisnis.

- **Create/Update:** Membersihkan input (misal: mengubah nama menjadi huruf kapital).
- **Read:** Memformat tampilan (misal: mengubah format tanggal).

2. Kategori Fungsi dalam Operasi CRUD

A. Fungsi String (Manipulasi Teks)

Sering digunakan dalam operasi **Create** dan **Update** untuk memastikan konsistensi format data.

Fungsi	Deskripsi	Contoh Penggunaan
UPPER() / LOWER()	Mengubah kapitalisasi	Menyeragamkan email menjadi huruf kecil.
CONCAT()	Menggabungkan teks	Menggabungkan nama_depan dan nama_belakang.
SUBSTRING()	Mengambil bagian teks	Mengambil 4 digit pertama kode barang.
TRIM()	Menghapus spasi liar	Membersihkan input nama dari spasi di awal/akhir.

Contoh Update:

```
UPDATE pelanggan
SET nama_pelanggan = UPPER(TRIM(nama_pelanggan))
WHERE id_pelanggan = 1;
```

B. Fungsi Tanggal dan Waktu (Date & Time)

Penting untuk pencatatan *timestamp* saat data dibuat atau diubah.

- **NOW() / CURRENT_TIMESTAMP**: Mendapatkan waktu saat ini.
- **DATEDIFF()**: Menghitung selisih hari antar dua tanggal.
- **DATE_FORMAT()**: Mengubah tampilan tanggal (Read).

Contoh Create (Insert):

```
INSERT INTO transaksi (id_transaksi, id_pelanggan, tgl_input)
VALUES (101, 1, NOW());
```

C. Fungsi Alur Kontrol (Conditional Functions)

Fungsi ini sangat berguna dalam operasi **Read** untuk memberikan informasi tambahan tanpa mengubah data asli.

- **COALESCE()**: Memberikan nilai pengganti jika data bernilai NULL.
- **IFNULL()**: (Spesifik MySQL) Mirip dengan COALESCE.
- **CASE WHEN**: Logika IF-THEN di dalam SQL.

Contoh Read dengan Logika:

```
SELECT nama_produk, stok,
    CASE
        WHEN stok = 0 THEN 'Habis'
        WHEN stok < 10 THEN 'Stok Menipis'
        ELSE 'Tersedia'
    END AS status_stok
FROM produk;
```

3. Implementasi Fungsi dalam Prosedur CRUD

1. READ (Format Data yang Lebih Baik)

Mengambil data dengan memformat angka mata uang dan tanggal agar mudah dibaca oleh pengguna.

```
SELECT
    id_transaksi,
    UPPER(nama_pelanggan) AS pelanggan,
    DATE_FORMAT(tgl_transaksi, '%d-%m-%Y') AS tanggal_format,
    CONCAT('Rp ', FORMAT(total_bayar, 0)) AS total_harga
FROM v_rekap_penjualan;
```

2. CREATE (Pembersihan Data Otomatis)

Memastikan data yang masuk ke database sudah dalam keadaan rapi (Normalisasi Teks).

```
INSERT INTO pelanggan (nama_pelanggan, email)
VALUES (TRIM(LOWER('  Fatimatuzzahra ')),
LOWER(fatimatuzzahra@polije.ac.id'));
```